

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Base de Datos II — Unidad 2: Optimización de Consultas

Trabajo Práctico — Semana 4

Reportes analíticos asistidos por IA sobre Food Store: joins, subconsultas, agregación y ventana

Grupo: Lautaro Gomez - Xavier Pappalardo - Nicolas Ibañez - Benjamin Arroyo - Maximiliano Reinoso - Martin Sisterna

Objetivos de la práctica

- Optimizar consultas analíticas lentas —con múltiples JOIN y agregación— sobre una base Food Store poblada masivamente.
- Usar la IA como motor primario para proponer reescrituras e índices a partir de planes reales de EXPLAIN ANALYZE.
- Contrastar la explicación de un plan de join generada por IA (Nested Loop, Hash Join, Merge Join) contra el plan real.
- Generar consultas resumen, rankings y subconsultas bajo especificación precisa, verificando la equivalencia de resultados.
- Documentar cada decisión en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store poblada masivamente, tal como quedó al cierre de la práctica de la Semana 3 (o repetir la carga si se trabaja sobre una copia nueva).
- Índices de la Semana 3 ya creados y medidos (idx_producto_categoria, idx_pedido_usuario, índice parcial de producto).
- Acceso a psql / DBeaver / pgAdmin y a la IA de la cátedra (OpenCode / Kiro).

Estado de la base al inicio de la práctica: El servidor local PostgreSQL 16 (localhost:5432) contenía la base food_store con el modelo completo de la Semana 3 (categoria, producto, usuario, pedido, detalle_pedido — con borrado lógico eliminado y los 3 índices de la Semana 3), pero no estaba poblada masivamente (5 usuarios, 5 pedidos, 8 detalles, 15 productos) y en las 5 tablas reltuples = -1 (nunca se ejecutó ANALYZE). Se decidió avanzar con la base disponible y documentar esta limitación en las mediciones. Las conclusiones de plan y la metodología son válidas; los tiempos deben releerse con esa salvedad.

Parte 1 — Laboratorio: consultas analíticas lentas

A diferencia de la Semana 3 —donde el cuello de botella era un filtro simple sobre una tabla—, acá el costo suele estar en cómo se combinan varias tablas y en el volumen que atraviesa cada JOIN antes de agregarse. Se eligieron consultas reales de queries.sql que cruzan al menos tres tablas del modelo.

Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.
3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

Consultas seleccionadas

ID	Consulta (origen en queries.sql)	Tablas cruzadas	Comandos de medición
Q1	Facturación por categoría y por mes — consulta analítica B de queries.sql	detalle_pedido → pedido → producto → categoria (4 tablas, 3 JOIN + agregación)	EXPLAIN (ANALYZE, BUFFERS)
Q2	Ranking de usuarios por gasto acumulado — consulta analítica C de queries.sql (versión que cruza 3 tablas: el gasto se agrega desde detalle_pedido)	usuario → pedido → detalle_pedido (3 tablas, 2 JOIN + agregación + función de ventana)	EXPLAIN (ANALYZE, BUFFERS)

Código SQL de las consultas seleccionadas:

```
-- Q1: Facturacion por categoria y mes (queries.sql B) — cruza 4 tablas
SELECT c.nombre_categoria AS categoria, date_trunc('month', ped.fecha_pedido) AS mes, SUM(dp.subtotal_detallepedido) AS
facturado
FROM detalle_pedido dp
JOIN pedido ped ON ped.id_pedido = dp.pedido_id AND ped.eliminado = FALSE
JOIN producto pr ON pr.id_producto = dp.producto_id
JOIN categoria c ON c.id_categoria = pr.categoria_id
WHERE dp.eliminado = FALSE
GROUP BY c.nombre_categoria, date_trunc('month', ped.fecha_pedido)
ORDER BY mes, facturado DESC;

-- Q2: Ranking de usuarios por gasto (queries.sql C, version que cruza 3 tablas)
SELECT u.id_usuario AS id, u.nombre_usuario || ' ' || u.apellido_usuario AS usuario, SUM(dp.subtotal_detallepedido) AS gasto,
       RANK() OVER (ORDER BY SUM(dp.subtotal_detallepedido) DESC) AS puesto
FROM usuario u
JOIN pedido ped ON ped.usuario_id = u.id_usuario AND ped.eliminado = FALSE
JOIN detalle_pedido dp ON dp.pedido_id = ped.id_pedido AND dp.eliminado = FALSE
WHERE u.eliminado = FALSE
GROUP BY u.id_usuario, u.nombre_usuario, u.apellido_usuario
ORDER BY puesto;
```

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
Q1 — Facturación por categoría y mes (4 tablas)	3 × Hash Join con estimaciones por defecto (nunca ANALYZE: rows=500 en pedido)	ANALYZE en las 5 tablas (reltuples -1 → reales) + CREATE INDEX idx_detalle_pedido_producto_id ON detalle_pedido(producto_id);	3 × Hash Join (el algoritmo no fue descartado por el optimizador con 7 filas)	6.775 ms → 0.301 ms (22.5x) — atribuible a ANALYZE
Q2 — Ranking de usuarios por gasto (3 tablas)	2 × Hash Join (estimaciones por defecto)	ANALYZE en las 5 tablas	2 × Hash Join (join order optimizado por las nuevas estadísticas)	1.509 ms → 0.455 ms (3.3x) — atribuible a ANALYZE

Criterio de aceptación aplicado: Ninguna propuesta se aplicó sin poder explicarla en defensa oral. La práctica prevé que agregar un índice “puede cambiar el algoritmo de join (por ejemplo, de Hash Join a Nested Loop)”;

acá el algoritmo no cambió y se documenta el motivo: con menos de 20 filas por tabla, construir y sondear un Hash es más barato que recorrer un B-tree (el índice agrega un nivel de indirección). El índice idx_detalle_pedido_producto_id queda justificado para la base masiva: al crecer detalle_pedido, el join conducido por pocas filas de producto podrá usar Nested Loop + Index Scan.

Planes de ejecución — Q1 (antes / después)

Plan ANTES (baseline)

```
Sort (cost=84.89..85.41 rows=210) (actual time=4.240..4.243 rows=7)
-> HashAggregate (Group Key: c.nombre_categoria, date_trunc(...))
-> Hash Join (cost=49.85..71.54) Hash Cond: (pr.categoria_id = c.id_categoria)
-> Hash Join (cost=37.38..57.44) Hash Cond: (dp.producto_id = pr.id_producto)
-> Hash Join (cost=26.25..45.75) Hash Cond: (dp.pedido_id = ped.id_pedido)
-> Seq Scan on detalle_pedido dp (est. rows=420, real 7)
-> Hash
-> Seq Scan on pedido ped (est. rows=500, real 4)
-> Hash
-> Seq Scan on producto pr (est. rows=50, real 15)
-> Hash
-> Seq Scan on categoria c (est. rows=110, real 6)
Execution Time: 6.775 ms / Planning Time: 15.731 ms
```

Nodo/join identificado: 3 Hash Join. Estimaciones 10x por falta de ANALYZE.

Plan DESPUÉS (ANALYZE + índice)

```
Sort (cost=4.97..4.99 rows=6) (actual time=0.177..0.179 rows=7)
-> HashAggregate (Group Key: c.nombre_categoria, date_trunc(...))
-> Hash Join (cost=3.42..4.74) Hash Cond: (pr.categoria_id = c.id_categoria)
-> Hash Join (cost=2.28..3.55) Hash Cond: (pr.id_producto = dp.producto_id)
-> Seq Scan on producto pr (rows=15)
-> Hash
-> Hash Join (cost=1.10..2.21) Hash Cond: (dp.pedido_id = ped.id_pedido)
-> Seq Scan on detalle_pedido dp (rows=7)
-> Hash
-> Seq Scan on pedido ped (rows=4)
-> Hash
-> Seq Scan on categoria c (rows=6)
```

Execution Time: 0.301 ms / Planning Time: 8.156 ms

El planificador mantiene Hash Join y no usa el nuevo índice (costo del camino indexado > Hash con 7 filas).

Planes de ejecución — Q2 (antes / después)

Plan ANTES (baseline)

```
Sort -> WindowAgg (rank)
-> Sort (sum gasto DESC)
-> HashAggregate (Group Key: u.id_usuario)
-> Hash Join (dp.pedido_id = ped.id_pedido)
-> Seq Scan detalle_pedido (est 420, real 7)
-> Hash
-> Hash Join (ped.usuario_id = u.id_usuario)
-> Seq Scan pedido (est 500, real 4)
-> Hash
-> Seq Scan usuario (est 30, real 4)
Execution Time: 1.509 ms
```

Plan DESPUÉS (ANALYZE)

```
Sort -> WindowAgg (rank)
-> Sort (sum gasto DESC)
-> HashAggregate (Group Key: u.id_usuario)
-> Hash Join (ped.usuario_id = u.id_usuario)
-> Hash Join (dp.pedido_id = ped.id_pedido)
-> Seq Scan detalle_pedido (rows=7)
-> Hash
-> Seq Scan pedido (rows=4)
-> Hash
-> Seq Scan usuario (rows=4)
Execution Time: 0.455 ms
```

Parte 2 — Lectura crítica de planes de join interpretados por IA

Los planes con varios JOIN son más ricos —y más fáciles de malinterpretar— que un Seq Scan simple. Se tomó el plan baseline de Q1 (Parte 1), con 3 nodos de join, y se le pidió a la IA que lo explique en lenguaje natural, nodo por nodo, sin más contexto que el texto del plan.

Explicación generada por la IA (reproducida textualmente)

“El plan comienza leyendo las 4 tablas con un nested loop: primero detalle_pedido, después pedido, producto y categoria. Para cada fila de pedido busca en detalle_pedido. Los tres joins son del tipo index nested loop. La tabla externa del join con pedido es pedido y la interna detalle_pedido, de la cual se construye el hash. La consulta termina agrupando por producto y mes en un HashAggregate. El costo estimado total, 84.89, son los milisegundos que va a tardar la consulta en el servidor. Se estima que leyó 500 filas de pedido.”

Contraste contra el plan real

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
“Los tres joins son index nested loop”	No	Los tres nodos de join son Hash Join (aparece textualmente “Hash Join ... Hash Cond”). No hay ningún Nested Loop.
“La tabla externa del join es pedido y la interna detalle_pedido (de la cual se construye el hash)”	No	En el nodo Hash Join (dp.pedido_id = ped.id_pedido) la tabla externa es detalle_pedido (Seq Scan on detalle_pedido dp) y la interna es pedido, sobre la que se construye el Hash. La IA invirtió externa/interna.
“El costo estimado total 84.89 son los milisegundos que tardará la consulta”	No	cost=84.89..85.41 es una unidad arbitraria del planificador. El tiempo real de ejecución fue 6.775 ms (Execution Time). Confundir cost estimado con tiempo real es el error típico que la consigna pide detectar.
“Se estima que leyó 500 filas de pedido”	No	500 es la estimación por defecto (tablas sin ANALYZE, reltuples = -1). El Seq Scan on pedido leyó realmente 4 filas (el filtro NOT eliminado descartó 1).
“Agrupa por producto y mes en un HashAggregate”	No	El HashAggregate agrupa por nombre_categoria y month(fecha_pedido) (Group Key: c.nombre_categoria, date_trunc(...)). No participa id/nombre de producto.
“La agregación ocurre después de completar los tres joins”	Sí	El nodo HashAggregate está por encima de los 3 Hash Join y agrega el resultado combinado de las 4 tablas (evidencia: indentación y Group Key).

Hallazgos: La IA falló en los 3 puntos que la consigna pide verificar con atención: (1) el tipo de join (Hash vs Nested Loop), (2) qué tabla es externa y cuál interna del join, y (3) confundir cost estimado con tiempo real; además error en el group key y en interpretar la estimación de filas como valores reales. La lección: contrastar SIEMPRE las afirmaciones contra el texto del plan.

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

Se practicó el flujo de especificar con precisión y verificar equivalencias sobre agregaciones, funciones de ventana y subconsultas correlacionadas — construcciones donde es fácil que dos consultas “parezcan” equivalentes sin serlo.

Se redactó una especificación precisa para dos consultas sobre Food Store: (a) un ranking con función de ventana y (b) una consulta con subconsulta correlacionada. La spec fija tablas, filtro de borrado lógico en cada tabla involucrada, columnas de salida, criterio de partición/orden y desempate.

(a) Ranking con función de ventana

*Spec: «Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada usuario vigente con al menos un pedido vigente, su nombre completo, el total gastado (suma de pedido.total en pedidos no eliminados) y su puesto en un ranking de mayor a menor gasto, sin colapsar filas. En caso de empate, deben compartir el mismo puesto. No uses SELECT *.»*

Versión A — RANK() de ventana sobre la agregación:

```
SELECT u.id_usuario, u.nombre_usuario || ' ' || u.apellido_usuario AS nombre_completo, SUM(p.total_pedido) AS total_gastado,
       RANK() OVER (ORDER BY SUM(p.total_pedido) DESC) AS puesto
```

```

FROM usuario u
JOIN pedido p ON p.usuario_id = u.id_usuario AND p.eliminado = FALSE
WHERE u.eliminado = FALSE
GROUP BY u.id_usuario, u.nombre_usuario, u.apellido_usuario;

```

Versión B — estructura distinta: agregación en CTE + ranking sin función de ventana (subconsulta que cuenta competidores con gasto estrictamente mayor + 1, que reproduce la semántica de RANK):

```

WITH gasto_usuario AS (
  SELECT u.id_usuario, u.nombre_usuario || ' ' || u.apellido_usuario AS nombre_completo, SUM(p.total_pedido) AS total_gastado
  FROM usuario u
  JOIN pedido p ON p.usuario_id = u.id_usuario AND p.eliminado = FALSE
  WHERE u.eliminado = FALSE
  GROUP BY u.id_usuario, u.nombre_usuario, u.apellido_usuario
)
SELECT gu.id_usuario, gu.nombre_completo, gu.total_gastado,
       (SELECT COUNT(*) + 1 FROM gasto_usuario gg WHERE gg.total_gastado > gu.total_gastado) AS puesto
FROM gasto_usuario gu;

```

(b) Subconsulta correlacionada (con búsqueda detallada de un bug)

*Spec: «Genera una consulta SQL sobre el esquema de Food Store que devuelva los productos vigentes cuya recaudación total (suma de subtotal de detalles no eliminados, dentro de pedidos no eliminados) supera el promedio de recaudación de los productos de su misma categoría. Incluir nombre del producto, categoría y recaudación. No uses SELECT *.»*

Bug detectado por EXCEPT (caso “parecen equivalentes pero no lo son”): La primera versión de la subconsulta correlacionada daba 0 filas, mientras la versión con funciones de ventana daba 2 filas. Causa raíz: el subquery agrupaba por categoría_id solamente (1 fila por categoría = TOTAL de la categoría) y AVG(recau) devolvía el total de la categoría entera; ningún producto superaba ese total. La comparación correcta exige el promedio por producto dentro de la categoría (GROUP BY categoría_id, id_producto). Este es exactamente el tipo de falla que la práctica busca: un JOIN dentro de la subconsulta sumaba filas y cambiaba la escala sin que ambas consultas “se notaran” distintas.

Versión A — subconsulta correlacionada (corregida):

```

SELECT pr.id_producto, pr.nombre_producto, c.nombre_categoria, SUM(dp.subtotal_detallepedido) AS recaudacion
FROM detalle_pedido dp
JOIN pedido ped ON ped.id_pedido = dp.pedido_id AND ped.eliminado = FALSE
JOIN producto pr ON pr.id_producto = dp.producto_id AND pr.eliminado = FALSE
JOIN categoria c ON c.id_categoria = pr.categoria_id AND c.eliminado = FALSE
WHERE dp.eliminado = FALSE
GROUP BY pr.id_producto, pr.nombre_producto, c.nombre_categoria, pr.categoria_id
HAVING SUM(dp.subtotal_detallepedido) > (
  SELECT AVG(pc.recaudacion)
  FROM (
    SELECT pr2.categoria_id, pr2.id_producto, SUM(dp2.subtotal_detallepedido) AS recaudacion
    FROM detalle_pedido dp2
    JOIN pedido ped2 ON ped2.id_pedido = dp2.pedido_id AND ped2.eliminado = FALSE
    JOIN producto pr2 ON pr2.id_producto = dp2.producto_id AND pr2.eliminado = FALSE
    JOIN categoria c2 ON c2.id_categoria = pr2.categoria_id AND c2.eliminado = FALSE
    WHERE dp2.eliminado = FALSE
    GROUP BY pr2.categoria_id, pr2.id_producto
  ) pc
  WHERE pc.categoria_id = pr.categoria_id
);

```

Versión B — estructura distinta: funciones de ventana (AVG OVER PARTITION BY categoria_id), sin subconsulta correlacionada:

```
WITH recau_producto AS (  
  SELECT pr.id_producto, pr.categoria_id, SUM(dp.subtotal_detallepedido) AS recaudacion  
  FROM detalle_pedido dp  
  JOIN pedido ped ON ped.id_pedido = dp.pedido_id AND ped.eliminado = FALSE  
  JOIN producto pr ON pr.id_producto = dp.producto_id AND pr.eliminado = FALSE  
  WHERE dp.eliminado = FALSE  
  GROUP BY pr.id_producto, pr.categoria_id  
)  
,  
con_promedio AS (  
  SELECT rp.id_producto, rp.categoria_id, rp.recaudacion,  
         AVG(rp.recaudacion) OVER (PARTITION BY rp.categoria_id) AS promedio_categoria  
  FROM recau_producto rp  
)  
SELECT cp.id_producto, pr.nombre_producto, c.nombre_categoria, cp.recaudacion  
FROM con_promedio cp  
JOIN producto pr ON pr.id_producto = cp.id_producto  
JOIN categoria c ON c.id_categoria = cp.categoria_id  
WHERE cp.recaudacion > cp.promedio_categoria;
```

Verificación de equivalencia (EXCEPT en ambos sentidos + conteos):

```
-- Ejemplo de verificación de equivalencia sobre un ranking  
-- (debe dar 0 filas en ambos sentidos si son equivalentes)  
(consulta_ranking_v1) EXCEPT (consulta_ranking_v2);  
(consulta_ranking_v2) EXCEPT (consulta_ranking_v1);
```

Verificación EXCEPT	Conteos de filas
rank v1 EXCEPT v2	0 filas
rank v2 EXCEPT v1	0 filas
rank conteo v1	3
rank conteo v2	3
sub v1 EXCEPT v2	0 filas
sub v2 EXCEPT v1	0 filas
sub conteo v1	2
sub conteo v2	2

Resultado obtenido: Ranking (v1 y v2): los puestos 1-2-3 coinciden (Juan Pérez 17300.00; Ana Garis 7300.00; Bruno Ledesma 4800.00). Subconsulta (v1 corregida y v2): ambas devuelven Muzzarella (9000.00, Pizzas) y Cerveza artesanal IPA (3600.00, Bebidas). Las cuatro consultas quedaron verificadas como equivalentes.

Parte 4 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta analítica integrante —con al menos dos JOIN y una agregación— y compiten por el mejor plan, con la IA como asistente. Cada equipo usa su asistente de IA

para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar en el plan real. Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE). Se documentan en la bitácora todas las propuestas de la IA que no funcionaron.

Consulta común de competencia:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT u.apellido_usuario || ' ' || u.nombre_usuario AS usuario, c.nombre_categoria, SUM(dp.subtotal_detallepedido) AS facturado
FROM detalle_pedido dp
JOIN pedido ped ON ped.id_pedido = dp.pedido_id AND ped.eliminado = FALSE
JOIN usuario u ON u.id_usuario = ped.usuario_id AND u.eliminado = FALSE
JOIN producto pr ON pr.id_producto = dp.producto_id
JOIN categoria c ON c.id_categoria = pr.categoria_id AND c.eliminado = FALSE
WHERE dp.eliminado = FALSE
GROUP BY u.id_usuario, u.apellido_usuario, u.nombre_usuario, c.id_categoria, c.nombre_categoria
ORDER BY facturado DESC; -- 4 tablas + agregacion: 4x Hash Join
```

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Gomez - Pappalardo - Ibañez - Arroyo - Reinoso - Sisterna	ANALYZE correctivo (estadísticas reales; el plan baseline trabajaba con reltuples=-1 y filas estimadas 10x) + índice idx_detalle_pedido_producto_id (reutilizado de la Parte 1, justificado en el nodo Hash Cond: dp.producto_id = pr.id_producto). Se descartó forzar join order (el planificador ya lo elige barato) y reescrituras con subconsultas (alteran semántica).	6.775	0.328	20.7x

Propuestas de IA evaluadas y no aplicadas (bitácora)

Propuesta de la IA	Resultado	Motivo
Reordenar joins forzando categoria de primera	Descartada	El planificador ya elige el join order más barato con las estadísticas corregidas por ANALYZE; ninguna evidencia en el plan la justifica.
Expresar totales con subconsultas correlacionadas para evitar el GroupAggregate	Descartada	Altera la semántica original (habría que re-especificar la consigna) y agrega un nivel de agregación redundante; el GroupAggregate no es un nodo caro en el plan.
Índice sobre pedido(fecha_pedido) para el GROUP BY mensual	Descartada	El plan no muestra ningún nodo Sort/Scan sobre fecha que lo justifique; en base masiva el date_trunc(mes) igual requiere sort.

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica. Herramientas: OpenCode / Big Pickle (asistente) y psql/DBeaver (medición). Una fila por uso relevante.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	Diagnosticar el plan baseline de Q1 y Q2	“Ejecuta EXPLAIN (ANALYZE, BUFFERS) de (B) facturación por categoría y mes y (C) ranking de usuarios por gasto de queries.sql; identifica el algoritmo de join de cada nodo.”	Aceptado: 3x Hash Join (Q1) y 2x Hash Join (Q2) + hallazgo reituples=-1 (nunca ANALYZE), evidenciado por estimaciones 10x.
OpenCode	Proponer cambios justificados en el nodo de join	“A partir del nodo ‘Hash Cond: dp.producto_id = pr.id_producto’ y de las estimaciones infladas, proponé ANALYZE y/o índices justificando cada uno en ese nodo.”	Aceptado: ANALYZE de las 5 tablas + idx_detalle_pedido_producto_id (no existía índice sobre la FK).
OpenCode	Re-medir y explicar el resultado	“Medí de nuevo el mismo script y explicá por qué el algoritmo de join no cambió.”	Aceptado como evidencia honesta: Hash Join se mantiene; con 7 filas construir Hash es más barato que un lookup indexado. Caso documentado según criterio de aceptación.
OpenCode	Parte 2: explicación nodo a nodo del plan de Q1	“Explicá este EXPLAIN en lenguaje natural, nodo por nodo, solo con el texto del plan.”	Contrastada: 4 afirmaciones incorrectas + 1 correcta detectadas contra el plan real (tipo de join, externa/interna, cost vs tiempo, group key).
OpenCode	Parte 3: generar SQL desde la spec de ranking	Spec textual del ranking de usuarios por gasto (ver Parte 3(a)).	Aceptado: v1 con RANK() y v2 con CTE + conteo de competidores; equivalentes (EXCEPT 0/0, conteos 3-3).
OpenCode	Parte 3: generar la subconsulta correlacionada desde spec	Spec de productos que superan el promedio de su categoría.	Primera versión DESCARTADA: fallaba la equivalencia (0 vs 2 filas); se corrigió el GROUP BY del subquery (se agregó id_producto). Caso documentado como ejemplo de “parecer equivalentes sin serlo”.
OpenCode	Parte 3: verificar equivalencia formal	“Comprobá con EXCEPT en ambos sentidos y conteos las 4 consultas.”	Aceptado: 0 filas en los 4 EXCEPT y conteos iguales.
OpenCode	Parte 4: optimizar la consulta de competencia	“Optimizá facturación por usuario y categoría (4 tablas + agregación); proponé reescrituras e índices justificados en el plan real.”	Aceptado: ANALYZE + índice ya justificado. Descartadas y bitacoreadas: forzar join order, reescrituras con subconsultas, índice sobre fecha_pedido.
Kiro	—	No fue utilizado en esta semana	—

Link al repositorio de Github: https://github.com/XavierPappalardo/UTN-TUPaD-TP_Base_de_Datos_Semana4_2PRO1

